

B-Safe CI/CD Pipeline

Final Specification Document — Course-End Project 3

1. Project and Tester Details

Project Title	B-Safe: CI/CD Pipeline for DevOps Transformation
Tester / Student Name	Okile Moses Otim
Course	Course-End Project 3 (SimpliLearn DevOps Capstone)
Background Scenario	Aetna (US healthcare) — 12-week legacy release cycle automated into a CI/CD pipeline
GitHub Repository	https://github.com/Ebosterix/b-safe-project-app
Docker Hub Repository	okile/b-safe-project-app (private)
Deployment Target	AWS EC2 instance ("Docker-node"), port 8090

2. Concepts Used in the Project

This project applies core DevOps and CI/CD concepts to convert a legacy, manual release process into an automated pipeline. The following concepts and tools were used:

Version Control (Git/GitHub): Single source of truth for application code, Dockerfile, test script, and pipeline definition (Jenkinsfile). Enables traceability of every change.

Jenkins Controller-Agent Architecture: Jenkins controller runs locally; an AWS EC2 instance is registered as a remote SSH agent. This separates orchestration from execution and mirrors real-world scalable Jenkins deployments.

Pipeline as Code: The entire build-test-push-deploy workflow is defined declaratively in a Jenkinsfile stored in the repository, rather than configured manually through the Jenkins UI.

Containerization (Docker): The application is packaged into a lightweight Docker image (nginx:alpine base) ensuring consistent behavior across build, test, and deployment environments.

Automated Testing: A smoke test script builds the image, runs a temporary container, and verifies both HTTP response code and page content before the pipeline is allowed to proceed.

Private Container Registry: Docker Hub is used as a private registry to store versioned, tagged images produced by each pipeline run, satisfying secure artifact storage requirements.

Secrets Management: Docker Hub credentials and the EC2 SSH private key are stored using Jenkins' built-in credentials store, never hard-coded into the Jenkinsfile or committed to source control.

Continuous Deployment: Each successful pipeline run automatically deploys the newly built image as a running container on the EC2 instance, exposed on a defined port for user access.

Infrastructure Cleanup: Post-build actions prune dangling Docker images after each run to prevent disk bloat from repeated builds.

3. Pipeline Stages Implemented

Stage	Purpose	Requirement Satisfied
Checkout	Pull latest code from GitHub (main branch)	Source control integration
Build Docker Image	Package the static app into an nginx:alpine image, tagged per build run image with artifacts	Req. #3 — build image
Run Automated Tests	Execute smoke_test.sh; halts pipeline on failure	Req. #4 — automated tests on build
Push to Docker Hub	Authenticate and push tagged image to private repository	Req. #5 — private repo + push
Deploy Application	Remove prior container (if any) and run new image on EC2	Req. #6 — expose app on port
Verify Deployment	curl the live port to confirm the app responds post-deploy	Deployment validation
Post: Prune Images	Remove dangling images after job completion	Req. #7 — remove container stack

4. Test Case Documentation

Test Case: Container Smoke Test (test/smoke_test.sh)

Objective: Verify the built Docker image runs correctly and serves the expected application content before it is pushed to the registry or deployed.

Algorithm:

- 1 Build a Docker image from the current Dockerfile, tagged as a temporary test image.
- 2 Start a container from the image, mapping host port 8090 to container port 80.
- 3 Wait 3 seconds for the Nginx process to initialize.
- 4 Send an HTTP GET request to the container and capture the status code and response body.
- 5 If the status code is not 200, print a failure message, clean up the container, and exit with code 1.
- 6 If the status code is 200, check the response body for the expected heading text.
- 7 If the expected text is missing, print a failure message, clean up, and exit with code 1.
- 8 If both checks pass, print a success message, clean up the container, and exit with code 0.

Execution Results:

Run	Environment	Result
-----	-------------	--------

Manual local run	Docker Desktop (local PC)	PASS — HTTP 200, content verified
Jenkins pipeline run	EC2 agent (b-safe-project-node)	PASS — pipeline proceeded to Push stage

5. Key Design Decisions

- Jenkins was kept on the local machine as the controller, with the EC2 instance registered as a remote SSH agent, rather than installing Jenkins directly on AWS — reducing resource load on the cloud instance while still satisfying the requirement to configure Jenkins architecture with AWS.
- Docker Hub was selected as the private registry over AWS ECR for simplicity and free-tier availability.
- Port 8090 was deliberately chosen for the deployed application to avoid conflicts with other services already running on the same EC2 instance.
- Requirement #7 ("remove container stack after completing the job") was interpreted as pruning build-time dangling images rather than removing the live running application container — since removing the running app would contradict requirement #6 (exposing the app for user access).
- Docker Hub credentials and the EC2 SSH key are stored exclusively in Jenkins' encrypted credentials store, never committed to source control. An exposed access token encountered during development was promptly revoked and rotated.

6. Conclusion and Unique Selling Points (USPs)

This project demonstrates a functioning, end-to-end CI/CD pipeline that replaces a slow, manual release process with an automated one: a single code push triggers build, test, packaging, registry push, and deployment without manual intervention. This directly addresses the business problem described in the brief — replacing a 12-week release cycle with a process capable of iterative, on-demand deployments.

Unique Selling Points:

- Fully automated release path: a single Git push results in a tested, versioned, deployed application with no manual steps.
- Built-in quality gate: automated smoke testing prevents broken builds from ever reaching the registry or production, reducing the risk of shipping defects.
- Resource-efficient Jenkins architecture: separating the controller (local) from the agent (cloud) reduces cloud compute costs while retaining full pipeline capability — a practical, cost-conscious pattern for small teams.
- Secure credential handling: sensitive tokens and keys are never exposed in code, logs, or version control, aligning with baseline security best practice.
- Reproducible builds: containerization guarantees the application behaves identically across local testing and production deployment, eliminating environment-specific defects.

Potential Enhancements:

- Add a staging environment and blue-green or rolling deployment strategy to eliminate downtime during redeploys.
- Introduce a monitoring/alerting layer (e.g. health checks, uptime monitoring) on the deployed container.
- Extend automated testing beyond smoke tests to include unit and integration tests as the application grows in complexity.
- Adopt token expiry and rotation policies for registry credentials as a stronger long-term security posture.
- Migrate to AWS ECR with IAM-scoped roles for tighter access control in a production-grade setting.

Full evidence (screenshots of Jenkins runs, Docker Hub repository, and the live deployed application) is available in the GitHub repository linked above.